Entornos de Desarrollo

Practica de IDEs

José Carlos Martín

FP Superior en Desarrollo de Aplicaciones Web

Curso 2025-2026

Granada, 5 de enero de 2026

Índice general

0.0.1 Comenzando a utilizar el IDE.. 4
0.1 Plugins o librerías instaladas 10
0.2 Código implementado 11

Índice de figuras

Figura Figura 1 Ventana Inicio Android Studio 4
Figura Figura 2 Seleccion de Plantilla 5
Figura Figura 3 Configuracion de nuestro proyecto 5
Figura Figura 4 Primera pantalla Android Studio 6
Figura Figura 5 Instalando Plugins 6
Figura Figura 6 Instalando Emulador Pixel Pro 9 7
Figura Figura 7 Primer Hello Android! emulado 7
Figura Figura 8 Estructura APK en JAVA 8
Figura Figura 9 Estructura Visual de una pagina 9
Figura Figura 10 Visualización Final 10

Introducción

En esta práctica se ha elegido el IDE **Android Studio** por su versatilidad, soporte multiplataforma y extensibilidad mediante plugins, ideal para desarrollo multiplataforma y tambien desarrollo web.

Para este pequeño proyecto se empleará repositorios sacados de Google oficiales y no oficiales para así poder entender mejor este IDE.

Todas las descargas, para poder seguir el guion de este proyecto son oficiales sin ningún tipo de riesgo.

Android Studio es el entorno de desarrollo integrado (IDE) oficial que se usa en el desarrollo de apps para Android. Basado en el potente editor de código y las herramientas para desarrolladores de IntelliJ IDEA, Android Studio ofrece aún más funciones que mejoran tu productividad cuando compilas apps para Android, como las siguientes:

- Un sistema de compilación flexible basado en Gradle
- Un emulador rápido y cargado de funciones
- Un entorno unificado donde puedes desarrollar para todos los dispositivos Android
- Ediciones en vivo para actualizar elementos componibles en emuladores y dispositivos físicos, en tiempo real
- Integración con GitHub y plantillas de código para ayudarte a compilar funciones de apps comunes y también importar código de muestra
- Variedad de marcos de trabajo y herramientas de prueba
- Herramientas de Lint para identificar problemas de rendimiento, usabilidad y compatibilidad de versiones, entre otros
- Compatibilidad con C++ y NDK

Desarrollo del trabajo

0.0.1. Comenzando a utilizar el IDE..

- Android Studio es una herramienta basada en Intelli J. Tiene de lenguaje de programación Java y Kotlin.
- La siguiente imagen despues de descargar e instalar el IDE nos aparecera esta ventana en la cual, tenemos que ir dandole click en los circulos rojos.

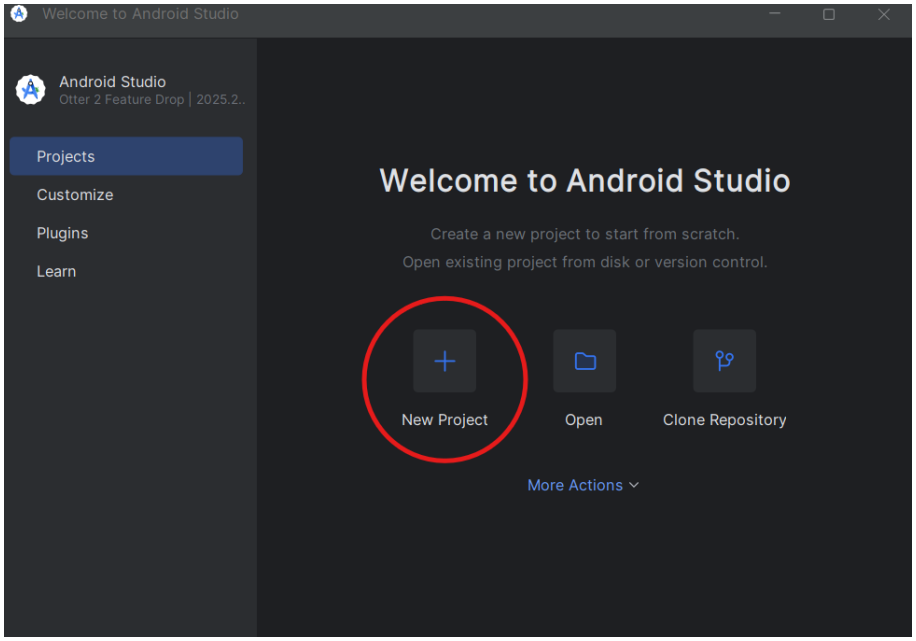

Figura 1: Ventana Inicio Android Studio

- En la siguiente ventana haremos click en **Empty Activity**, esta ventana nos muestra las opciones prediseñadas que tenemos a modo de plantilla para poder comenzar a desarrollar. Como nosotros solo vamos a ver a modo de fundamentos el IDE, seleccionamos la que he definido para una app sin menu en modo pagina en blanco.
- De todas formas, si nosotros quisieramos esta plantilla podriamos verla como por asi decirlo un plantilla en blanco con la que trabajar desde 0.

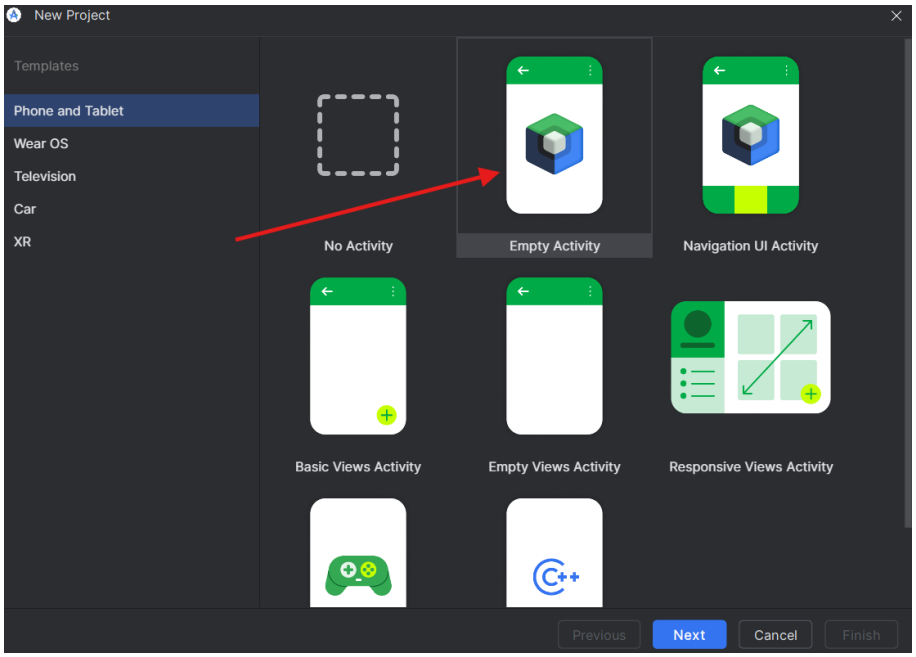

Figura 2: Seleccion de Plantilla

En esta pagina siguiente nos encontramos con la configuración que deseamos darle a nuestro proyecto. Tendremos lo siguiente:

- Nombre del proyecto
- Localizacion del proyecto
- Version de ANDROID minima que seleccionamos.
- Y lenguaje de construccion de la app que deseamos.

Como estamos dando Java actualmente, sera el lenguaje que vamos a desarrollar. Igualmente tenemos tambien la opcion de trabajar con Kotlin (si lo dominas es mejor).

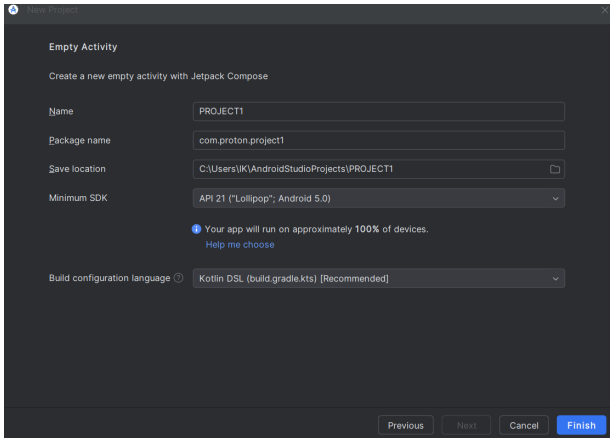

Figura 3: Configuracion de nuestro proyecto

Como se puede apreciar en la imagen en Android Studio el entorno de código es igual que el de Intelli J entonces sera algo mas familiar. En la parte izquierda nos encontraremos

con la estructura de nuestro proyecto la cual veremos más adelante. En la parte derecha tendremos las herramientas de **Grandle** es basicamente nuestro emulador y testeador de la app. Y tambien nos encontraremos la mayoria de herramientas posicionadas en esta parte. Y en la zona central como podemos observar tendremos el codigo del archivo que estemos trabajando.

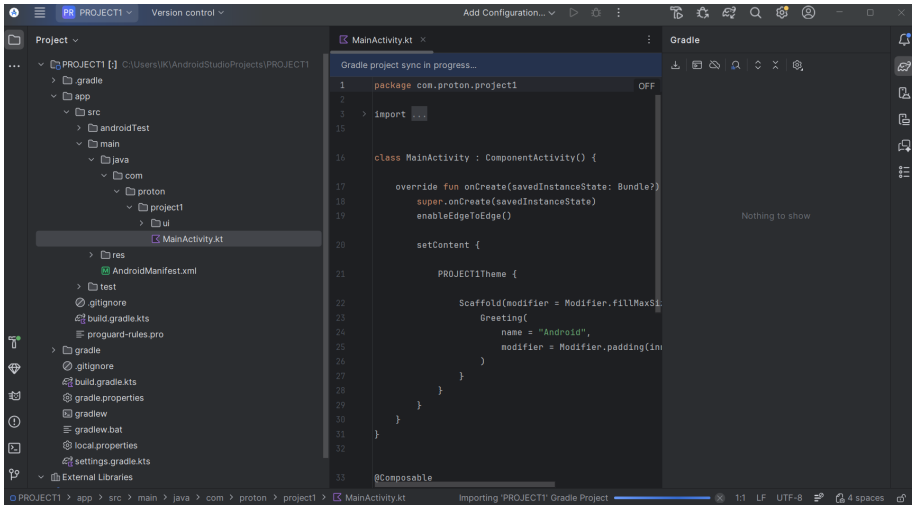

Figura 4: Primera pantalla Android Studio

Lo primero que he hecho yo es cambiar a idioma español el IDE , para ello hemos accedido a settings y buscamos en plugins el paquete de idioma :

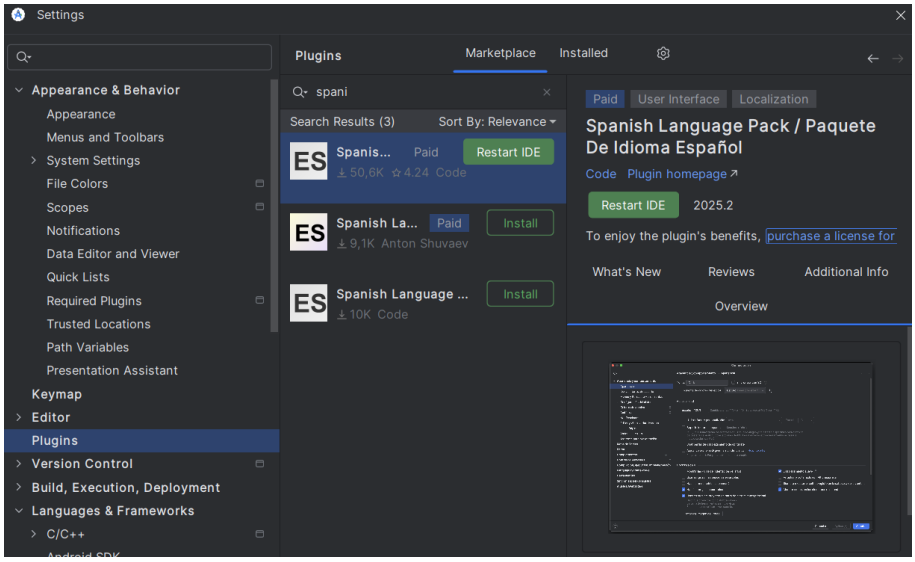

Figura 5: Instalando Plugins

Pero nos avisa de que es bajo suscripción , ya que Android Studio tiene programa de suscripción para poder utilizar los plugins.

Ahora vamos a crear un emulador para reproducir y testear nuestras apps en Android Studio. Tendrás una gran variedad de dispositivos , pero mi elección ha sido , el Pixel Pro 9. Para todos , la primera vez tendrás que descargar el SDK, que viene siendo el sistema Android. Una vez seleccionado dejamos que se descargue y configure nuestro emulador:

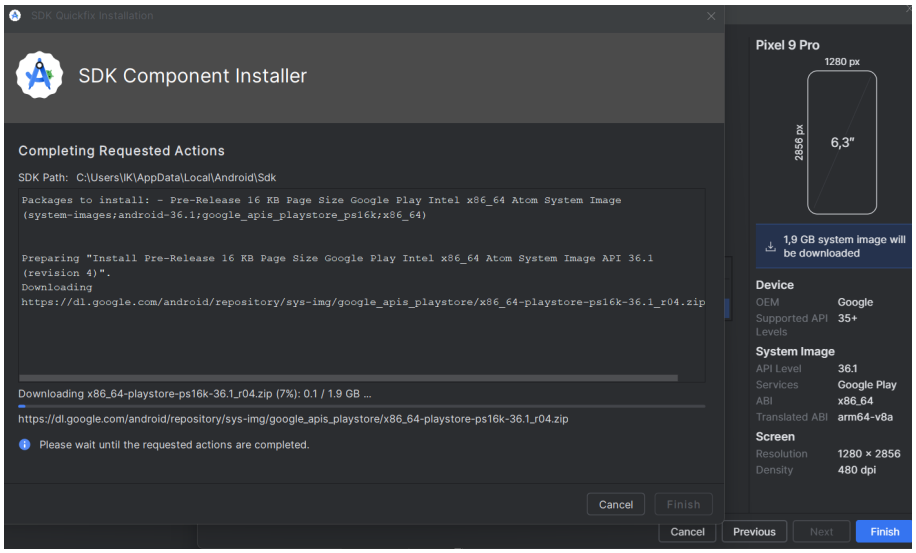

Figura 6: Instalando Emulador Pixel Pro 9

Esto tardara un rato pero al acabar nos saldra nuestro "Hello Android!"que viene por defecto con nuestra plantilla en blanco que seleccionamos.

Figura 7: Primer Hello Android! emulado

Para este proyecto ahora que conocemos como funciona a modo de introducción Android Studio, vamos a irnos a la parte del código. Como dato , hay que decir que en Android Studio no solo se trabaja con Kotlin (mayoria de veces) , si no que tambien se trabaja con Java (si quieres). Entonces pasemos a la estructura que tiene nuestra app:

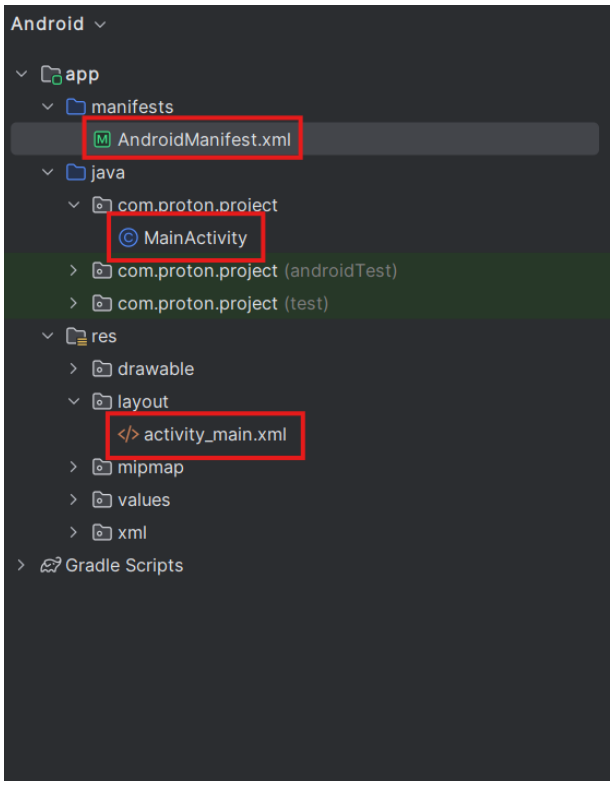

Figura 8: Estructura APK en JAVA

1. AndroidManifest. Este archivo ubica la configuración principal de la app de Android que estamos creando, nos dara la version , configuración de pantallas de nuestra app y nombre que tiene nuestra app en el menu HOME de Android en el telefono.

2. Main Activity En este archivo nos referimos a la pantalla principal de nuestra app, el cual tenemos alojado nuestro codigo de nuestra pantalla principal.

3. Layout / activity main Corresponde a nuestro archivo de edicion de la pantalla principal. (Por asi decirlo su CSS).

Para cada archivo activity tendremos un layout con el mismo nombre, este correspondera a su mismo activity.

Ahora abriendo el layout podremos empezar a trabajar con la pantalla principal de nuestra app, como veis tenemos nuestro Hello Android , el cual vamos a modificar y podremos poner lo que nosotros queramos. Android Studio ahora nos facilitara una grandisima variedad de opciones para personalizar a nuestro gusto la app que queramos.

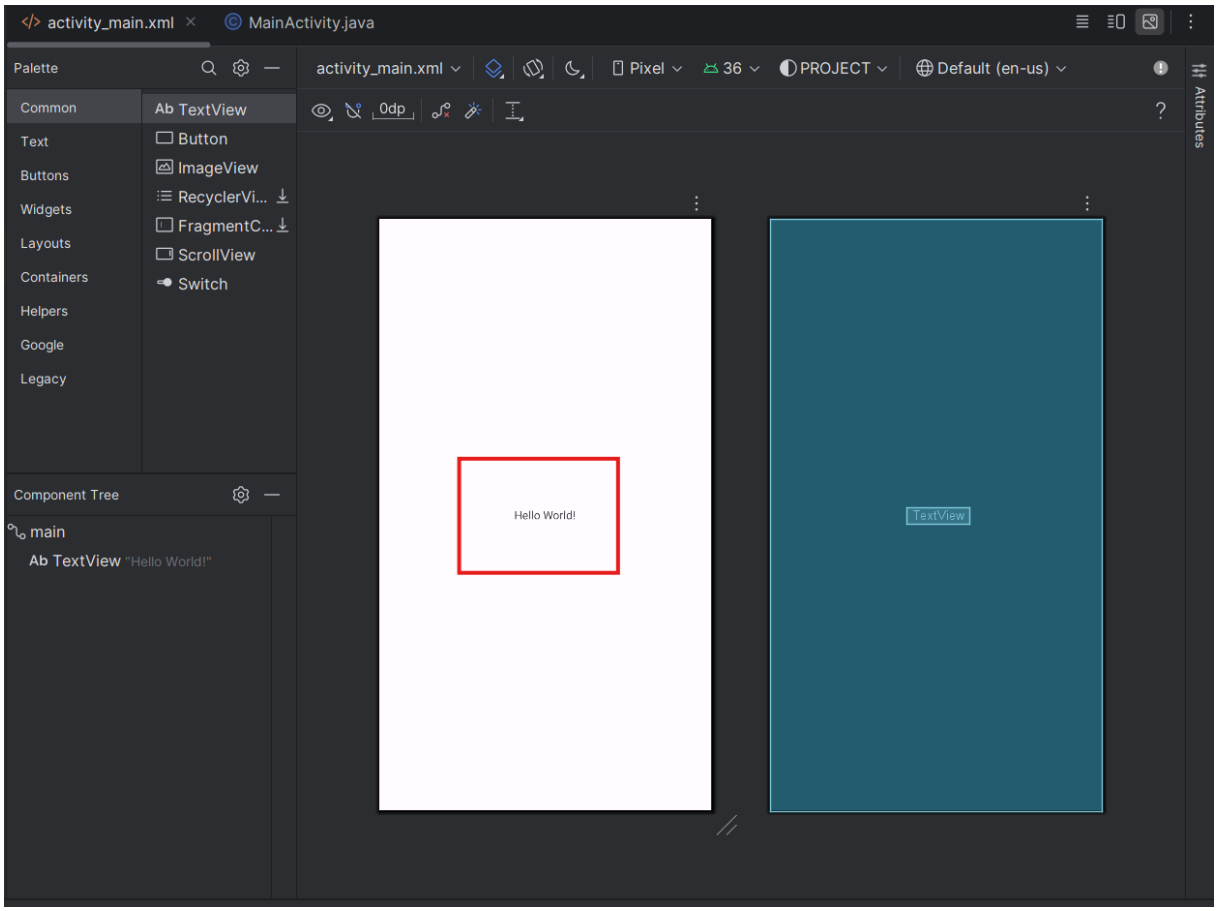

Figura 9: Estructura Visual de una pagina

Y utilizando las herramientas que nos proporciona Android Studio, hemos creado esta app que solo tendria una pagina principal y tiene un semaforo de botones y un calendario para así probar los widgets. La app no tiene ninguna funcionalidad pero nos ayuda como plantilla y así apartir de aqui desarrollar el resto del proyecto a futuro que nosotros queramos.

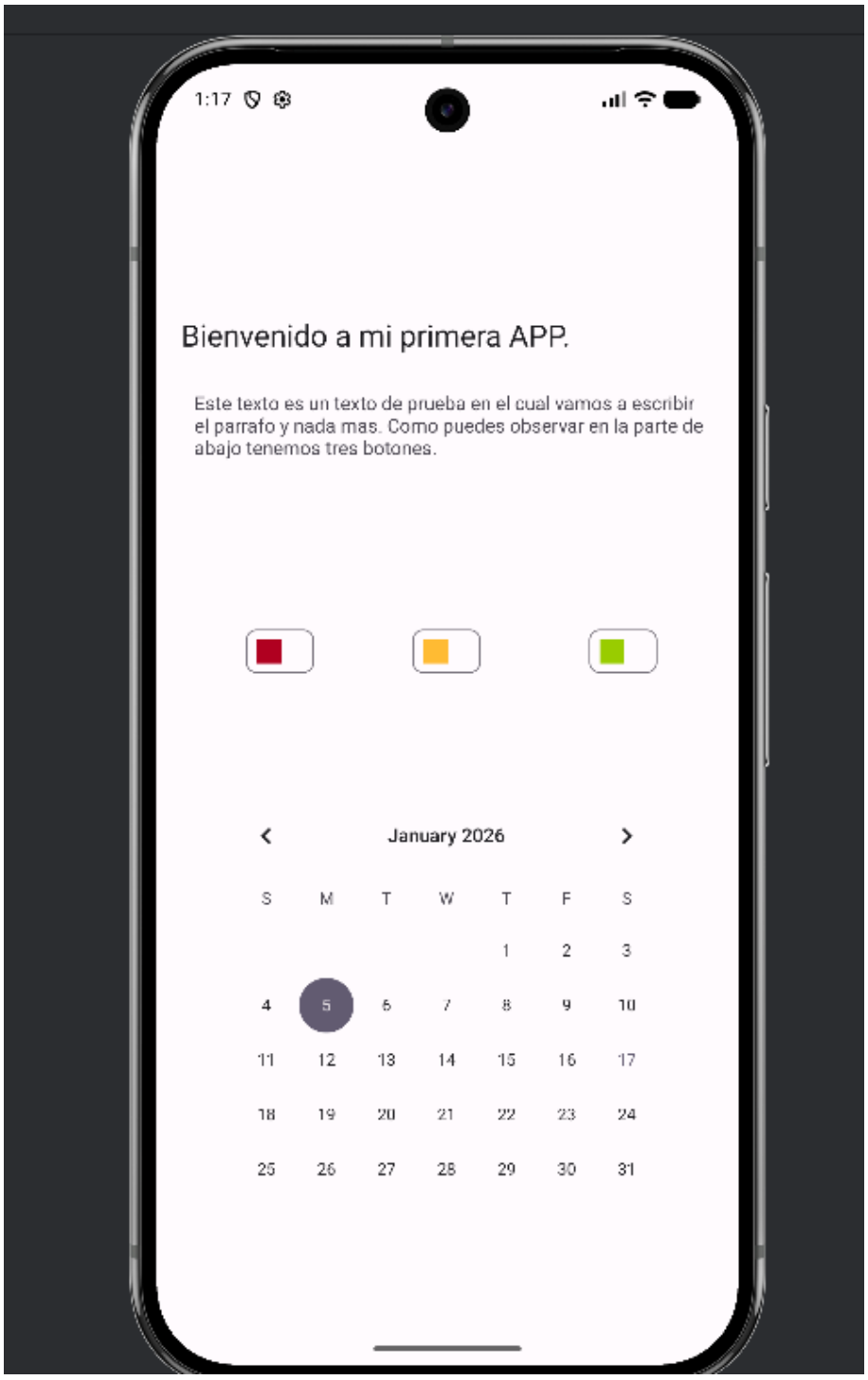

Figura 10: Visualización Final

0.1. Plugins o librerías instaladas

Estas son las librerias que genera Android Studio automáticamente en dependencies ... para una app mínima como la que tenemos:

- implementation 'androidx.core:core-ktx:1.13.1' (o versión actual): Extensiones Kotlin para Java, útiles para tareas comunes.

- implementation 'androidx.appcompat:appcompat:1.7.0': Soporte para temas y componentes UI en versiones antiguas de Android.
- implementation 'com.google.android.material:material:1.12.0': Componentes Material Design (botones, cards, etc.).
- implementation 'androidx.constraintlayout:constraintlayout:2.1.4': Layout flexible para diseños responsivos.
- Pruebas: testImplementation 'junit:junit:4.13.2' y androidTestImplementation 'androidx.test.ext:junit:1.2.1'.

Estos son los plugins que me he instalado, me he guiado por ser utiles para trabajo continuado:

- Key Promoter X (5.5M+): Muestra atajos de teclado para acciones con ratón, acelerando el flujo de trabajo.
- Material Theme UI (14.6M+): Temas Material Design para el IDE, reduce fatiga visual.
- SonarLint (7.1M+): Análisis en tiempo real de bugs, seguridad y calidad de código.
- ADB Idea (1.3M+): Comandos ADB rápidos (reiniciar app, limpiar datos) sin terminal.
- GitToolBox: Mejora Git con estados en tiempo real, branches y atajos.
- CodeGlance: Mini-mapa del código para navegar archivos largos.
- Rainbow Brackets: Colorea paréntesis y llaves para código legible.

0.2. Código implementado

En esta sección voy a exponer los bloques de codigo de cada elemento de mi pequeña app de prueba:

Listing 1: **Código Text View**

```
<TextView
  android:id="@+id/textView2"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:text="@string/bienvenido_a_mi_primera_app"
  android:textAppearance="@style/TextAppearance.AppCompat.Large"
  app:layout_constraintBottom_toBottomOf="parent"
  app:layout_constraintEnd_toEndOf="parent"
  app:layout_constraintHorizontal_bias="0.13"
  app:layout_constraintStart_toStartOf="parent"
  app:layout_constraintTop_toTopOf="parent"
  app:layout_constraintVertical_bias="0.155" />
```

Listing 2: **Código Botones**

```
<com.google.android.material.chip.Chip
        android:id="@+id/chip2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:chipIcon="@android:color/holo_orange_light"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.498"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.427" />
```

Listing 3: **Código Widget Calendario**

```
<CalendarView
        android:id="@+id/calendarView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.491"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.89" />
```

Conclusiones

Para finalizar este proyecto, app básica en Android Studio con Java demuestra un dominio sólido de conceptos fundamentales como Fragments, ViewPager2, layouts XML y lógica interactiva.

La forma de programar con IntelliJ es bastante intuitiva y incorporarla en Android Studio, facilita mucho la forma en que los desarrolladores nos sentimos mas comodos.

Ademas, el emulador de Gradle es muy completo, con un soporte estupendo y variedad de versiones SDK + formato de pantalla y telefonos grandisima. (estan casi todos).